

Data Science and AI for Finance

DA6701

Case study: Insurance Pricing

Nandan Sudarsanam

Department of Data Science and AI
Wadhvani School of Data Science and AI
Indian Institute of Technology Madras

- Context and environment.
- Problem statement.
- Challenges in solving the problem statement.
- Discuss two approaches to solving the problem.

Insurance terms at a glance

Term	What It Is
1. Price/Premium	The cost you pay (monthly/yearly) to keep the policy active.
2. Claim/ pay-out	Your formal request to the company to pay for a loss/ actual amount paid
3. Deductible	The amount you must pay out-of-pocket before the insurer pays.
4. Co-pay	A percentage on the claimed and sanctioned amount.
5. Sum Assured	The maximum amount the insurance company will pay for a loss.
6. Exclusion	Specific conditions or items that are NOT covered.
7. Rider	An optional add-on to customize your coverage.

Risk Management

- How would you trade off between deductible, co-pay and sum assured?
- To make “Expected Loss” equal (\$1,000) while creating different preferences, we manipulate the **Frequency** (Probability) and **Severity** (Claim Amount) of the events.
- Here are your three candidates:
 - **The “High Frequency” User (Person A):** 100% chance of losing \$1,000.
 - *Scenario:* Routine maintenance, dental cleanings, minor GP visits.
 - *Preference:* **Low Deductible.** Since they know they will claim, a high deductible eats their entire benefit.
 - **The “Balanced” User (Person B):** 10% chance of losing \$10,000.
 - *Scenario:* A broken leg, a minor surgery, or a car fender-bender.
 - *Preference:* **Low Co-pay.** The claim is big enough to clear the deductible easily, but a high co-pay (e.g., 20% of \$10k) would cost them \$2,000 — double their expected loss!
 - **The “Black Swan” User (Person C):** 1% chance of losing \$100,000.
 - *Scenario:* Major cancer treatment, total car loss, house fire.
 - *Preference:* **High Sum Assured.** Deductibles and co-pays are irrelevant if the bill is \$100k and the insurance caps out at \$50k. They need the “tail risk” covered.

Landscape: Market efficiency & Premium components

- **The Market Objective:** Insurance markets strive for **Risk Homogeneity**. Ideally, every policyholder pays exactly their expected loss ($E[L]$) + expense load + Profit.
- **The Cross-Subsidization Problem:** What happens when low-risk users overpay to cover high-risk users and vice versa.
 - *Cause:* Inability to observe variables (Asymmetric Information) or Regulatory caps.
 - *Solution:* ML/Statistical segmentation reduces the “Noise” ϵ in $Premium = f(Risk) + \epsilon$
- **Types of Insurance & Specific Factors:**
 - **P&C (Auto/Home):** Frequency (Accidents/Fires) $\times$ Severity (Repair Cost).
 - **Health:** Utilization Rate (Doctor Visits) $\times$ Cost per Service.
 - **Life:** Mortality Tables (Static) + Lifestyle/IoT (Dynamic).
- **The ML Opportunity Zone:**
 - **Linear (GLM):** Good for Age, Gender, Territory (Monotonic risks).
 - **Non-Linear (GBM/Neural Nets):** Good for:
 - **Interactions:** Age $\times$ CarPower $\times$ Weather.
 - **Telematics/IoT:** Driving behavior, steps taken, smart home sensor data.
 - **High-Cardinality:** 5,000+ ICD-10 medical codes or Vehicle Trims.

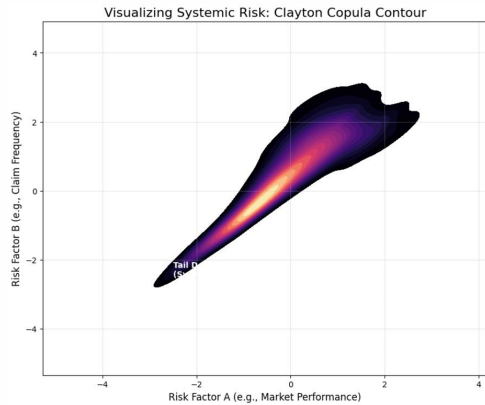
Systemic Risk & Dependence (The Copula)

- **Why Independence Fails:**

- Standard assumption:

$$P(\text{Claim}_A \cap \text{Claim}_B) = P(\text{Claim}_A) \cdot P(\text{Claim}_B)$$

- **Systemic Factors:** A hurricane, pandemic, or economic recession affects *all* policies simultaneously. This is **Tail Dependence**.
- **Frequency–severity correlation:** Use copulas when claim count and claim size show residual dependence beyond covariates; improves expected cost and tail risk.



The Copula Function (C)

- A tool to “couple” marginal distributions $F_1(x_1)$ and $F_2(x_2)$ into a joint distribution $H(x_1, x_2)$ while preserving correlation structure.
- **Sklar's Theorem:** Any multivariate joint distribution can be expressed as a copula applied to its marginal distributions.

$$H(x_1, \dots, x_d) = C(F_1(x_1), \dots, F_d(x_d))$$

- **Use Case:** Capital modeling. We don't price individual premiums with Copulas; we use them to calculate the **Solvency Capital Requirement (SCR)** to ensure survival in tail scenarios.

Modeling Problem

- We want to avoid **Pricing error**. We know that the **Premium** depends on expected payout.
- **Objective:** Create a model to predict expected payout.
- The model can be used as a black box, or explainability around the model can be used to create business rules.
- **Challenge:** Training data on payout has a whole bunch of zeros (people didn't file claims), and for the remaining it resembles a continuous variable with significant variability (cost of claims is different).
- **Implication:** Off-the-shelf supervised learning algorithms fail to build good models on this target variable.
- We will discuss two solutions: **Dual Modelling** vs **Single Tweedie**.

Dual Modeling Approach

- **Decomposition:** Model expected premium base as

$$E[\text{loss}] = E[\text{frequency}] \cdot E[\text{severity}]$$

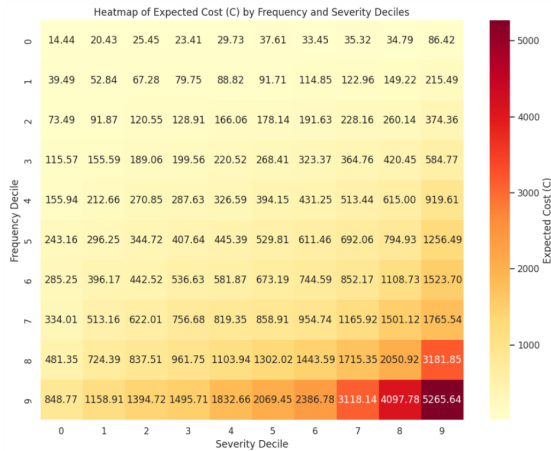
- **Frequency model:** GLM/GBM for claim count with exposure offset; Poisson or negative binomial (for overdispersion, variance $>$ mean, which is typical in insurance data).
- **Severity model:** GLM/GBM on positive amounts (Gamma/Lognormal) conditioned on at least one claim.
- **Dependence layer:** Optional copula between frequency and severity residuals; or hierarchical random effects across cohorts.
- **Cohort formation:** Use monotone transformations of predicted frequency and severity to create risk grids (e.g., deciles $\times$ deciles), then merge cells with similar expected cost and stability.
- **Fair pricing:** Set cohort premium proportional to $E[\text{frequency}] \cdot E[\text{severity}]$, apply loadings for expense/target margin; monitor cohort migration to avoid cross-subsidization drift.

Dual Modeling Approach: Cohort Formation

- **Risk grid:** Build a 2D matrix of frequency score deciles vs severity score deciles; each cell has $\hat{\lambda}$ and $\hat{\mu}$, with expected cost

$$\hat{C} = \hat{\lambda} \cdot \hat{\mu}$$

- **Stability filters:** Minimum exposure per cell, minimum claim count, confidence intervals on $\hat{C}$; merge or split cells accordingly.
 - Can we make a case to not focus on $\hat{C}$, but keep frequency and severity separate?



Cohort Governance & Monitoring

- **Regulatory overlays:** Enforce constraints (non-discrimination, explainability, caps, monotonicity for protected classes).
- **Interpretability tools:** SHAP/partial dependence plots to explain cohort boundaries; monotone constraints on key features to avoid pricing inversions.
- **Calibration:** Apply calibration layers per cohort (Platt-style or isotonic for probability; log-link calibration for severity).
- **Monitoring:** Drift and fairness dashboards; trigger re-segmentation if cohort loss ratios diverge beyond thresholds.

Single Model with Tweedie

- A Tweedie distribution with the power parameter p between 1 and 2 is equivalent to a **compound Poisson–Gamma distribution**.
- This means observations arise from a **Poisson process** (number of claims), where each event has a **Gamma-distributed amount** (severity), creating a **single model** that handles both **zero occurrences** and **continuous positive values** — perfect for modeling **insurance claims**.

Canonical moments:

$$E[Y] = \mu, \quad \text{Var}(Y) = \phi\mu^p, \quad 1 < p < 2$$

Compound Poisson–Gamma representation (for $1 < p < 2$):

$$Y = \sum_{i=1}^N Z_i, \quad N \sim \text{Poisson}(\lambda), \quad Z_i \sim \text{Gamma}(\alpha, \theta)$$

Parameter relationships:

Tweedie Parameter Relationships

For $1 < p < 2$:

$$\lambda = \frac{\mu^{2-p}}{\phi(2-p)}$$

$$\alpha = \frac{2-p}{p-1}$$

$$\theta = \phi(p-1)\mu^{p-1}$$

Interpretation:

- λ controls expected claim count.
- α and θ govern severity shape and scale.
- Power parameter p links frequency and severity behavior.

Comparison: Tweedie vs Dual

Aspect	Dual Approach	Tweedie Approach
Model Structure	Separate frequency and severity models	Single compound Poisson–Gamma model
Interpretability	High (decile grids, SHAP, separate insights)	Moderate (compound structure less intuitive)
Zero Claims Handling	Explicit frequency modeling	Inherent via Poisson component
Cross-Subsidization Risk	Low (explicit cohort control)	Higher (hidden within single model)
Regulatory Transparency	High	Moderate
Calibration Needs	Frequency + severity calibration	Single calibration layer
Deployment Complexity	Moderate (two models)	Lower (single pipeline)

Decision Lens:

- Choose **Tweedie** for compact modeling.
- Choose **Dual** for pricing fairness and cohort control.

Gradient Boosted Decision Trees (GBDT)

Goal: Build an additive model that minimizes loss.

$$\hat{y}(x) = \sum_{t=1}^T f_t(x)$$

where $f_t(x)$ is the prediction from the t -th decision tree.

Step 1: Initialize model

Choose constant prediction minimizing loss:

$$\hat{y}^{(0)} = \arg \min_c \sum_i L(y_i, c)$$

For regression this is typically the **mean of targets**.

Boosting Idea

Sequentially add trees that reduce the loss function.

$$\hat{y}^{(t)}(x) = \hat{y}^{(t-1)}(x) + f_t(x)$$

GBDT Training Step

For each boosting iteration $t = 1 \dots T$:

Step 2: Compute gradients (pseudo-residuals)

$$g_i = \frac{\partial L(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i}$$

Residuals:

$$r_i = -g_i$$

Step 3: Fit regression tree

Train a tree $f_t(x)$ to predict residuals r_i .

Step 4: Update predictions

$$\hat{y}^{(t)} = \hat{y}^{(t-1)} + \eta f_t(x)$$

Final model

$$\hat{y}(x) = \sum_{t=1}^T \eta f_t(x)$$

LightGBM: Optimization Improvements

LightGBM improves Gradient Boosted Decision Trees by using **second-order optimization**.

Gradients and Hessians

Instead of only gradients, LightGBM uses both:

$$g_i = \frac{\partial L}{\partial \hat{y}_i}, \quad h_i = \frac{\partial^2 L}{\partial \hat{y}_i^2}$$

Leaf value computation

Using a Newton update:

$$v = -\frac{\sum g_i}{\sum h_i + \lambda}$$

where λ is a regularization parameter.

This second-order approximation allows:

- Faster convergence
- More accurate loss minimization
- Better split evaluation

LightGBM: Faster Tree Construction

1. Histogram-based split finding

Continuous features are bucketed into bins.

$$G_b = \sum_{i \in b} g_i, \quad H_b = \sum_{i \in b} h_i$$

Split gain is computed as:

$$Gain = \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}$$

2. Leaf-wise tree growth (best-first)

- Traditional GBDT: level-wise expansion
- LightGBM: split the leaf with **maximum gain**
- Produces deeper trees with fewer iterations

Adopting LGBM to Tweedie

Step	Standard GBDT (Gaussian/MSE)	Tweedie GBDT (Insurance/Deviance)
1. Initialization	$F_0(x) = \text{mean}(y)$	$F_0(x) = \frac{\sum w_i y_i}{\sum w_i}$ (Weighted Mean)
2. Loss Function	$L = \frac{1}{2}(y - \hat{y})^2$	$L = -y \frac{e^{(1-p)F(x)}}{1-p} + \frac{e^{(2-p)F(x)}}{2-p}$
3. Gradient (g_i)	$g_i = \hat{y}_i - y_i$	$g_i = -y_i e^{(1-p)F(x)} + e^{(2-p)F(x)}$
4. Hessian (h_i)	$h_i = 1$ (constant)	$h_i = -y_i(1-p)e^{(1-p)F(x)} + (2-p)e^{(2-p)F(x)}$
5. Leaf Split	$\max \frac{(\sum g_i)^2}{\sum h_i + \lambda}$	$\max \frac{(\sum g_i)^2}{\sum h_i + \lambda}$ (variance info in h_i)
6. Update	$F_m(x) = F_{m-1}(x) + \eta \text{Tree}_m(x)$	$F_m(x) = F_{m-1}(x) + \eta \text{Tree}_m(x)$